Planning and Reinforcement Learning in AI Systems

(PARLAI)

Lesson 7a - Model-free RL (Part 3)

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology

Gradient Descent

Gradient Descent

Given a function $f(\theta_1, ..., \theta_n)$ of n real values $\theta = (\theta_1, \ldots, \theta_n)$, we want to minimize f with respect to θ

Gradient Descent

- The gradient of f at point θ, denoted $\nabla f(\theta)$ is an n-dimensional vector that points in the direction where f increases most steeply at point θ.
- Calculus tells us that $\nabla f(\theta)$ is just a vector of partial derivatives

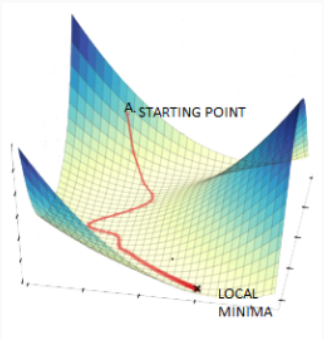
$$\nabla f(\theta) = \left[\frac{\partial f(\theta)}{\partial \theta_1}, \ldots, \frac{\partial f(\theta)}{\partial \theta_n} \right]$$

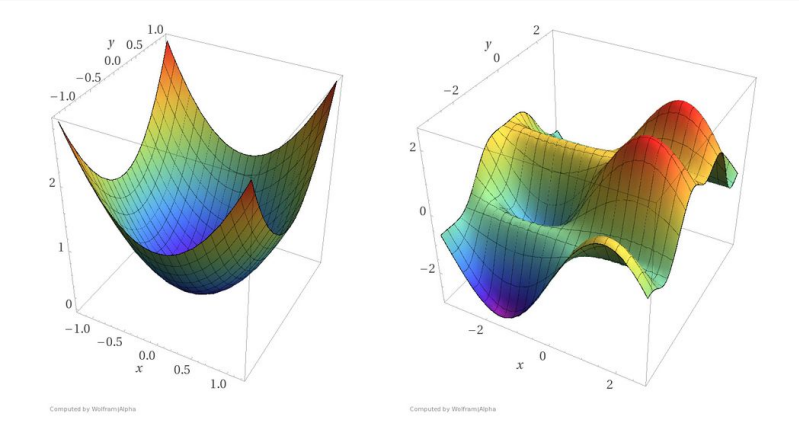
where $\frac{\partial f(\theta)}{\partial \theta_i} = \lim_{\epsilon \to 0} \frac{f(\theta_1, ..., \theta_{i-1}, \theta_i + \epsilon, \theta_{i+1}, ..., \theta_n) - f(\theta)}{\epsilon}$

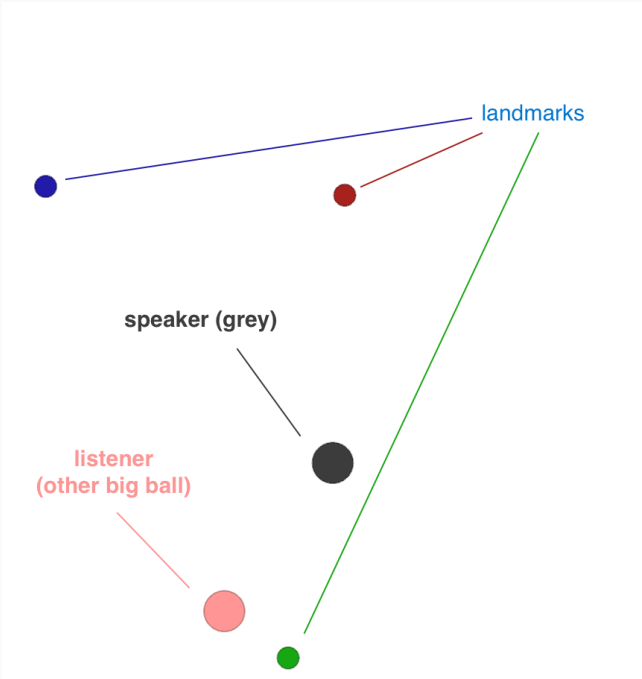
We can decrease f by moving in negative gradient direction

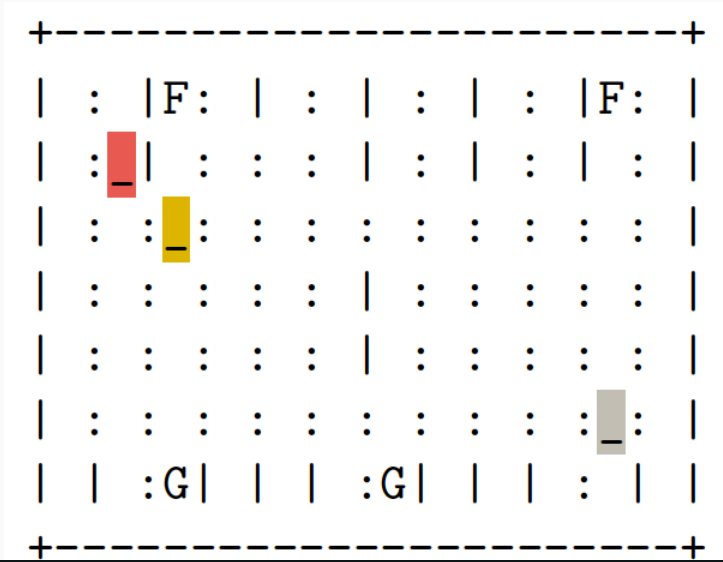
Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Descent









Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Visual Example: Gradient Descent

Suppose we want to minimize

$$f(\theta) = (\theta - 3)^2.$$

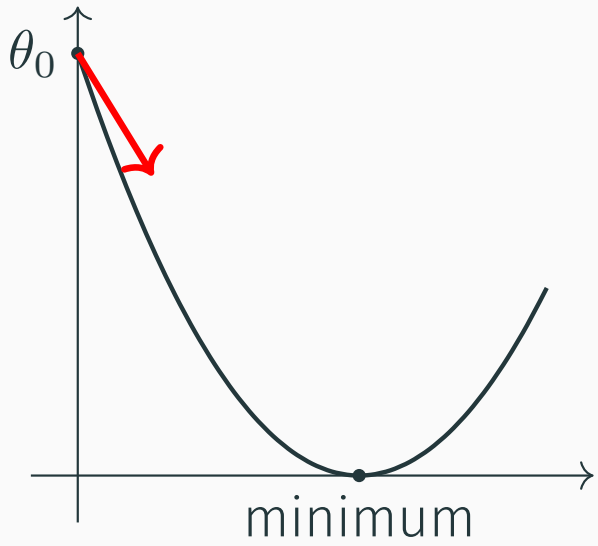
The gradient is

$$\frac{df}{d\theta} = 2(\theta - 3).$$

If we start with $\theta = 0$, then $\nabla f(0) = -6$
Gradient descent performs

$$\theta \leftarrow 0 - \alpha(-6) = 0 + 6\alpha$$

where α is the learning rate.
The parameter moves toward the minimum.



Gradient descent repeatedly moves in the direction of steepest decrease.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Descent for Squared Error

- What is our target (expected) value for state s?

- Our goal is to

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Descent for Squared Error

- What is our target (expected) value for state s?
- Suppose that we have a sequence of states and target values for each state $\langle s_1, V(s_1)\rangle, \langle s_2, V(s_2)\rangle, \ldots$
  - e.g, TD-signal, episode return, etc.
- Our goal is to ?

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Descent for Squared Error

- What is our target (expected) value for state s?
- Suppose that we have a sequence of states and target values for each state $\langle s_1, V(s_1)\rangle, \langle s_2, V(s_2)\rangle, \ldots$
  - e.g, TD-signal, episode return, etc.
- Our goal is to -> minimize the sum of squared errors between our estimated function and each target value:

$$\mathbb{E}_j = \frac{1}{2}(V_\theta(s_j) - V(s_j))^2$$

  where
  - $\mathbb{E}_j$ is the squared error of example j
  - $V_\theta(s_j)$ is our estimated value for s_j
  - $V(s_j)$ is the target of s_j
- For s_j ?

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Descent for Squared Error

- What is our target (expected) value for state s?
- Suppose that we have a sequence of states and target values for each state $\langle s_1, V(s_1)\rangle, \langle s_2, V(s_2)\rangle, \ldots$
  - e.g, TD-signal, episode return, etc.
- Our goal is to -> minimize the sum of squared errors between our estimated function and each target value:

$$\mathbb{E}_j = \frac{1}{2}(V_\theta(s_j) - V(s_j))^2$$

where

  - $\mathbb{E}_j$ is the squared error of example j
  - $V_\theta(s_j)$ is our estimated value for s_j
  - $V(s_j)$ is the target of s_j
- For s_j -> the gradient descent rule tells us that we can decrease error by updating parameters by:

$$\theta_i := \theta_i - \alpha \frac{\partial \mathbb{E}_j}{\partial \theta_i}$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Descent for Squared Error

We update every parameter using gradient descent:

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial E_j}{\partial \theta_i}.$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Descent for Squared Error

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

We update every parameter using gradient descent:

$$\theta_i \leftarrow \theta_i - \alpha \frac{\partial E_j}{\partial \theta_i}.$$

The parameter θ_i does not affect the error directly.

Instead,

$$\boxed{\theta_i \longrightarrow V_\theta(s_j) \longrightarrow E_j}$$

Changing a parameter changes the predicted value, which in turn changes the error. Therefore, we use the **chain rule** from calculus:

$$\frac{\partial E_j}{\partial \theta_i} = \underbrace{\frac{\partial E_j}{\partial V_\theta(s_j)}}_{\text{How does the error change?}} \cdot \underbrace{\frac{\partial V_\theta(s_j)}{\partial \theta_i}}_{\text{How does the prediction change?}}$$

The chain rule breaks the derivative into smaller ones.

Deriving the Gradient Update

The prediction error for example j is

$$E_j = \frac{1}{2} \left(V_\theta(s_j) - V(s_j) \right)^2 .$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Deriving the Gradient Update

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

The prediction error for example j is

$$E_j = \frac{1}{2}\left(V_\theta(s_j) - V(s_j)\right)^2.$$

The first derivative is

$$\frac{\partial E_j}{\partial V_\theta(s_j)} = V_\theta(s_j) - V(s_j)_j,$$

which is simply

$$\boxed{\text{prediction} - \text{target}}$$

Deriving the Gradient Update

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

For a linear value function $V_\theta(s) = \sum_i \theta_i f_i(s)$, we have

$$\frac{\partial V_\theta(s)}{\partial \theta_i} = f_i(s).$$

where $f_i(s)$ is the feature function. The gradient descent rule becomes

$$\boxed{\theta_i \leftarrow \theta_i + \alpha(V(s_j) - V_\theta(s_j)) f_i(s_j)}$$

Each parameter is updated according to:

learning rate $\times$ prediction error $\times$ feature value.

Policy-based RL

From Value-Based to Policy-Based RL

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

In value-based methods, we learn a value function:

$$V^\pi(s) \quad \text{or} \quad Q^\pi(s,a)$$

and then derive a policy, for example:

$$\pi(s) = \arg\max_a Q(s,a)$$

In policy-based methods, we learn the policy directly:

$$\pi_\theta(a \mid s)$$

Instead of learning how good actions are, learn how to act.

Reminder: Policy

- A **deterministic** policy is a mapping:
  - $\pi : \mathcal{S} \rightarrow \mathcal{A}$ from states to actions
  - $\pi : \mathcal{B} \rightarrow \mathcal{A}$ from beliefs to action
  - $\pi(s)$/ $\pi(\beta)$ is the (single) action the agent will perform at state s or belief β.
- A **non-deterministic** policy is a mapping:
  - $\pi : \mathcal{S} \rightarrow \mathcal{A}^n$ from states to actions,
  - $\pi : \mathcal{B} \rightarrow \mathcal{A}^n$ from beliefs to actions
  - $\pi(s)$/ $\pi(\beta)$ is the set of actions, one of which the agent will perform at state s or belief β.
- A **stochastic** policy is a mapping:
  - $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ from states to actions
  - $\pi : \mathcal{B} \times \mathcal{A} \rightarrow [0, 1]$ from beliefs to action
  - $\pi(s, a)$/ $\pi(\beta, a)$ is the probability the agent will perform action a at state s/ belief β.

Why do we need a probabilistic policy?

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Deterministic vs. Stochastic Policies

A deterministic policy maps states directly to actions:

$$a = \pi(s).$$

A stochastic policy defines a distribution over actions:

$$\pi(a \mid s).$$

Example:

$$\pi(\text{left} \mid s) = 0.4, \qquad \pi(\text{right} \mid s) = 0.6.$$

Policy-gradient methods usually optimize stochastic policies.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Why Learn the Policy Directly?

Policy-based methods are especially useful when:

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Why Learn the Policy Directly?

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy-based methods are especially useful when:

- The action space is continuous.
- The optimal behavior should be stochastic.
- Computing $\arg\max_a Q(s, a)$ is difficult.
- We want smooth changes in behavior.
- We use neural networks to output action distributions.

Parameterized Policies

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

We represent the policy using parameters θ:

$$\pi_\theta(a \mid s)$$

The parameters may be:

- entries in a table,
- weights of a linear model,
- weights of a neural network.

The learning problem becomes:

Find parameters θ that produce high-return behavior.

Example: Softmax Policy

Assume one state and two actions:

$$a \in \{L, R\}.$$

Let the policy use action preferences:

$$\theta = \begin{bmatrix} \theta_L \\ \theta_R \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

The softmax policy is:

$$\pi_\theta(a) = \frac{e^{\theta_a}}{e^{\theta_L} + e^{\theta_R}}.$$

Therefore:

$$\pi_\theta(L) = \frac{e^2}{e^2 + e^1} \approx 0.73, \qquad \pi_\theta(R) = \frac{e^1}{e^2 + e^1} \approx 0.27.$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy Gradient

The Policy Objective

We want to maximize expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[G(\tau) \right].$$

A trajectory is:

$$\tau = s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_T.$$

The discounted return from time t is:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}.$$

Learning means changing θ to increase $J(\theta)$.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Gradient Decent / Ascent

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

If we want to maximize $J(\theta)$, we use gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta).$$

Alternatively, we might aim at minimizing a loss measure (and perform gradient descent):

$$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta).$$

Move parameters in the direction that increases expected return.

Gradient Decent / Ascent

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

The policy-gradient trick uses:

$$\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s).$$

This lets us write the gradient of expected return as:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right].$$

Increase the probability of actions that produced high return.

Policy-Gradient Update Rule

For one sampled transition in an episode:

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Interpretation:

- If G_t is large and positive, make a_t more likely in s_t.
- If G_t is negative, make a_t less likely in s_t.
- The term $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ tells us how to change parameters to affect the selected action.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Example: Setup

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

We use the same softmax policy:

$$\theta = \begin{bmatrix} \theta_L \\ \theta_R \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

So:

$$\pi(L) = 0.73, \qquad \pi(R) = 0.27.$$

Assume the agent samples action $a = R$ and later receives return: $G = 8$

How should the policy change? How do we know if 8 is good? How do we know if R is better than L?

Example: Setup

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

We use the same softmax policy:

$$\theta = \begin{bmatrix} \theta_L \\ \theta_R \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

So:

$$\pi(L) = 0.73, \qquad \pi(R) = 0.27.$$

Assume the agent samples action $a = R$ and later receives return: $G = 8$

How should the policy change? How do we know if 8 is good? How do we know if R is better than L?

Example: Gradient of a Softmax Log-Policy

The gradient gives the direction for increasing the probability of the selected action. The return G determines how strongly we move in that direction. The learning rate α controls the size of the step.

For a softmax policy, if action a was selected, we can compute the gradient of the log-probability of that action and change the policy according to:

$$\frac{\partial \log \pi(a)}{\partial \theta_i} = \begin{cases} 1 - \pi(i), & i = a, \\ -\pi(i), & i \neq a. \end{cases}$$

Here, the selected action is R. Therefore:

$$\nabla_\theta \log \pi(R) = \begin{bmatrix} -\pi(L) \\ 1 - \pi(R) \end{bmatrix} = \begin{bmatrix} -0.73 \\ 0.73 \end{bmatrix}.$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Example: Compute the Gradient Update

The policy-gradient update is:

$$\theta \leftarrow \theta + \alpha G \nabla_\theta \log \pi(R).$$

Use:

$$\alpha = 0.1, \qquad G = 8.$$

Then:

$$G \nabla_\theta \log \pi(R) = 8 \begin{bmatrix} -0.73 \\ 0.73 \end{bmatrix} = \begin{bmatrix} -5.84 \\ 5.84 \end{bmatrix}.$$

So:

$$\theta_{new} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} + 0.1 \begin{bmatrix} -5.84 \\ 5.84 \end{bmatrix} = \begin{bmatrix} 1.416 \\ 1.584 \end{bmatrix}.$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Example: Effect on the Policy

Before the update:

$$\theta = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad \Rightarrow \quad \pi(L) = 0.73, \quad \pi(R) = 0.27.$$

After the update:

$$\theta = \begin{bmatrix} 1.416 \\ 1.584 \end{bmatrix}.$$

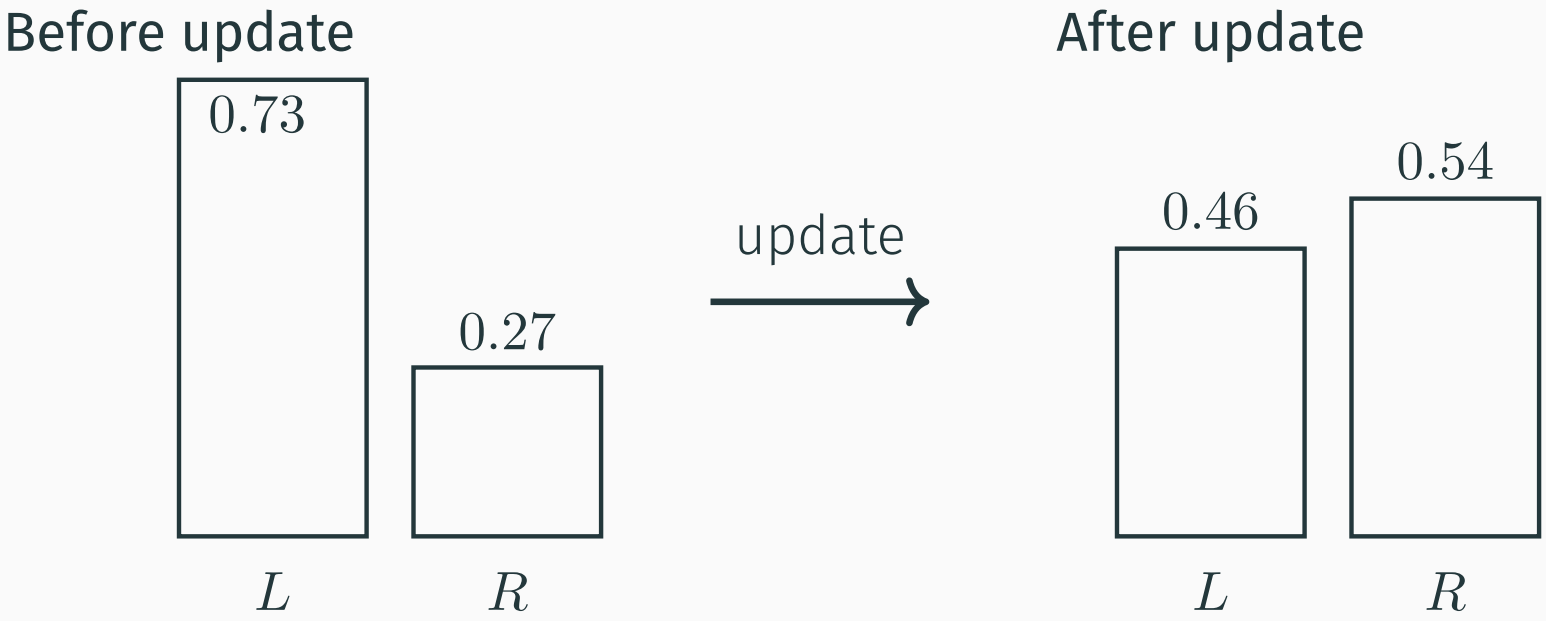
New softmax probabilities:

$$\pi(L) \approx 0.46, \qquad \pi(R) \approx 0.54.$$

The successful action R became more likely.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Example:Visualization of the Policy Change



The gradient does not directly edit probabilities. It changes parameters, and the new parameters induce new probabilities.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

What if the Return Is Negative?

Suppose the same action R was selected, but now:

$$G = -6.$$

The update becomes:

$$\theta \leftarrow \theta + \alpha(-6)\nabla_\theta \log \pi(R).$$

This moves parameters in the opposite direction.

Positive return reinforces the action.
Negative return discourages the action.

The only reason learning still works here is that, over many samples, L will also be examined, and if it receives even larger updates than R, it will lead to the larger updates in its direction. Nevertheless, we will see ways to address the issues that came up here.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

REINFORCE

Policy Gradient: A General Framework

How can we use policy gradients to learn a policy?

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy Gradient: A General Framework

How can we use policy gradients to learn a policy?

All policy-gradient methods follow the same basic idea:

At every iteration:

1. Execute the current policy
   $$\pi_\theta(a|s)$$
2. Collect trajectories
   $$\tau = s_0, a_0, r_1, \ldots$$
3. Estimate how good each selected action was
   $$G_t, \qquad Q(s,a), \qquad A(s,a)$$
4. Update the policy
   $$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

Different policy-gradient algorithms differ only in Step 3.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy Gradient: A General Framework

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

There is a variety of approaches. The main difference between algorithms is in **how they estimate the quality of each action.**

Algorithm	Uses
REINFORCE	Monte-Carlo return G_t
Actor-Critic	TD error / Advantage
PPO	Clipped advantage estimate
SAC	Soft Q-value + entropy

Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

It answers Step 3 by using the Monte-Carlo return:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

It answers Step 3 by using the Monte-Carlo return:

$$\boxed{G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}}$$

The update becomes

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(a_t|s_t).$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy-Gradient Algorithm: REINFORCE

REINFORCE is the simplest policy-gradient algorithm.

It answers Step 3 by using the Monte-Carlo return:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$$

The update becomes

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(a_t|s_t).$$

REINFORCE = policy gradient + Monte-Carlo estimate of action quality.

It is called a Monte Carlo method because G_t is computed from sampled returns.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

REINFORCE Pseudocode

REINFORCE

Initialize policy parameters θ
while not converged **do**
    Generate an episode using π_θ
    for each time step t in the episode **do**
        Compute return $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}$
        $\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
    end for
end while

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

The Main Problem: High Variance

Policy-gradient estimates can be very noisy. This causes unstable gradient estimates.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

The Main Problem: High Variance

Policy-gradient estimates can be very noisy. This causes unstable gradient estimates.

We can reduce variance by subtracting a baseline $b(s_t)$:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[(G_t - b(s_t))\nabla_\theta \log \pi_\theta(a_t \mid s_t)\right].$$

A common choice is:

$$b(s_t) = V(s_t).$$

Then:

$$G_t - V(s_t)$$

is an estimate of the advantage.

The baseline reduces variance without changing the expected gradient.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Actor-Critic

From REINFORCE to Actor-Critic

REINFORCE estimates the return using complete episodes.

Limitations:

- Must wait until the episode terminates.
- High-variance Monte Carlo estimates.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

From REINFORCE to Actor-Critic

REINFORCE estimates the return using complete episodes.

Limitations:

- Must wait until the episode terminates.
- High-variance Monte Carlo estimates.

Instead, we can learn a function that estimates the return while interacting with the environment.

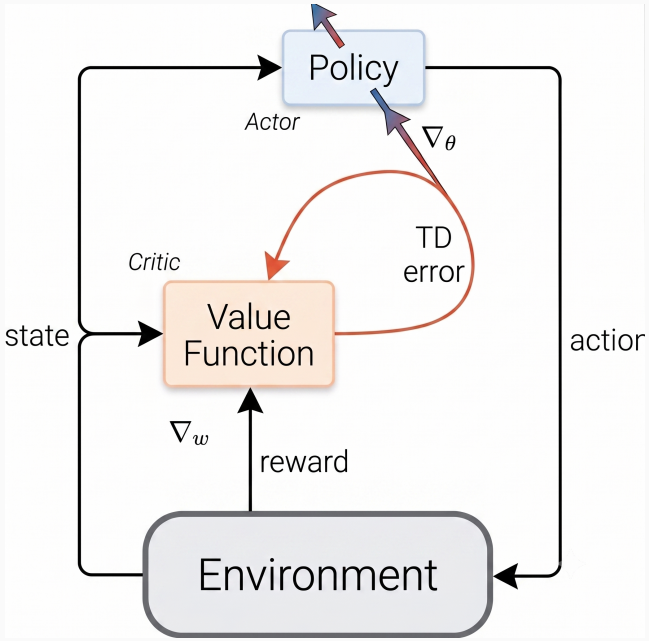
Actor-Critic methods learn two functions:

- **Actor**: policy $\pi_\theta(a|s)$
- **Critic**: value function $V_w(s)$ (or $Q_w(s,a)$)

The critic evaluates actions while the actor learns the policy.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

How Actor-Critic Works



The interaction loop is:

1. Actor selects an action.
2. Environment returns reward and next state.
3. Critic evaluates the outcome.
4. Actor updates the policy using the critic's feedback.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

How Actor-Critic Works

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Adding a value-function baseline to REINFORCE already gives an Actor-Critic architecture.

However, if the critic is still trained from complete Monte Carlo returns, we still suffer from delayed updates.

A common approach is to train the critic with TD learning, allowing updates after every interaction.

Actor-Critic Algorithm

One-Step TD Actor-Critic

Initialize actor parameters θ
Initialize critic parameters w
while not converged **do**
    Observe current state s
    Sample action $a \sim \pi_\theta(\cdot|s)$
    Execute action
    Observe reward r and next state s'
    Compute TD error

$$\delta = r + \gamma V_w(s') - V_w(s)$$

    Update critic

$$w \leftarrow w + \beta\delta\nabla_w V_w(s)$$

    Update actor

$$\theta \leftarrow \theta + \alpha\delta\nabla_\theta \log \pi_\theta(a|s)$$

end while

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Advantage

The advantage measures how much better (or worse) an action is compared to the average action in the same state.

$$A(s, a) = Q(s, a) - V(s).$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Advantage

The advantage measures how much better (or worse) an action is compared to the average action in the same state.

$$A(s, a) = Q(s, a) - V(s).$$

Suppose the critic predicts $V(s) = 8$.

This means that, on average, we expect a return of 8.

Now suppose taking action a actually gives $Q(s, a) = 11$.

Then $A(s, a) = Q(s, a) - V(s) = 3$ (the action was better than expected).

Alternatively, $Q(s, a) = 6$ gives $A(s, a) = -2$, i.e., the action was worse than expected.

The advantage measures whether an action performed better or worse than expected.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s,a) = Q(s,a) - V(s).$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s,a) = Q(s,a) - V(s).$$

The critic does not know the true action value $Q(s,a)$.

Instead, it approximates it using one-step TD:

$$Q(s,a) \approx r + \gamma V(s').$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s,a) = Q(s,a) - V(s).$$

The critic does not know the true action value $Q(s,a)$.

Instead, it approximates it using one-step TD:

$$Q(s,a) \approx r + \gamma V(s').$$

Therefore,

$$A(s,a) \approx r + \gamma V(s') - V(s).$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Where Does the Advantage Estimate Come From?

The actor should update according to the advantage

$$A(s,a) = Q(s,a) - V(s).$$

The critic does not know the true action value $Q(s,a)$.

Instead, it approximates it using one-step TD:

$$Q(s,a) \approx r + \gamma V(s').$$

Therefore,

$$A(s,a) \approx r + \gamma V(s') - V(s).$$

This TD estimate replaces the Monte Carlo return in REINFORCE, enabling updates after every interaction.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Advantage Actor-Critic (A2C)

A2C combines:

- an **actor** that learns the policy,
- a **critic** that estimates the value function.

Instead of waiting until the end of an episode, the actor updates after every interaction using the critic's estimate of the advantage:

$$\hat{A}_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

The actor update is

$$\theta \leftarrow \theta + \alpha \hat{A}_t \nabla_\theta \log \pi_\theta(a_t|s_t).$$

The critic provides immediate feedback, greatly reducing variance compared to REINFORCE.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Advantage Actor-Critic (A2C)

One-Step Advantage Actor-Critic

Initialize actor parameters θ
Initialize critic parameters w
while training **do**
    Observe state s
    Sample action $a \sim \pi_\theta(\cdot|s)$
    Execute a
    Observe reward r and next state s'
    Compute
    $$\delta = r + \gamma V_w(s') - V_w(s)$$
    Critic update
    $$w \leftarrow w + \beta\delta\nabla_w V_w(s)$$
    Actor update
    $$\theta \leftarrow \theta + \alpha\delta\nabla_\theta \log \pi_\theta(a|s)$$
end while

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Why Isn't A2C Enough?

Although A2C is much more efficient than REINFORCE, it still has a problem.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Why Isn't A2C Enough?

Although A2C is much more efficient than REINFORCE, it still has a problem.

Large policy updates may change the policy too much.

This can cause

- unstable learning,
- catastrophic drops in performance,
- oscillation between good and bad policies.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Why Isn't A2C Enough?

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Although A2C is much more efficient than REINFORCE, it still has a problem.

Large policy updates may change the policy too much.

This can cause

- unstable learning,
- catastrophic drops in performance,
- oscillation between good and bad policies.

Can we improve the policy while preventing excessively large updates?

Proximal Policy Optimization (PPO)

PPO is an Actor-Critic algorithm with one additional idea:

Don’t let the policy change too much in a single update.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Proximal Policy Optimization (PPO)

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

PPO is an Actor-Critic algorithm with one additional idea:

Don't let the policy change too much in a single update.

Instead of taking arbitrarily large gradient steps,

PPO clips the policy update so that

$$\pi_{\text{new}} \approx \pi_{\text{old}}.$$

Many small, safe improvements are better than a few large, unstable ones.

Proximal Policy Optimization (PPO)

Idea: Improve the policy gradually without changing it too much at once.

Repeat:

1. Collect trajectories using the current policy.
2. Estimate the advantage $\hat{A}_t$.
3. Compute the policy gradient.
4. Limit the size of the policy update using **clipping**.
5. Update the actor and critic.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Proximal Policy Optimization (PPO)

Idea: Improve the policy gradually without changing it too much at once.

Repeat:

1. Collect trajectories using the current policy.
2. Estimate the advantage $\hat{A}_t$.
3. Compute the policy gradient.
4. Limit the size of the policy update using **clipping**.
5. Update the actor and critic.

The probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$$

measures how much the new policy changes the probability of taking action a_t in state s_t.

PPO prevents the policy from changing too much in a single update, making learning much more stable

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

PPO Objective

Without any restriction, gradient ascent may make very large policy updates, causing performance to collapse.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

PPO Objective

Without any restriction, gradient ascent may make very large policy updates, causing performance to collapse.

PPO therefore maximizes

$$L^{\mathrm{PPO}} = \min\Big(r_t(\theta)\hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\Big).$$

where

$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$.

The clipping function restricts

$$1-\epsilon \leq r_t(\theta) \leq 1+\epsilon,$$

preventing excessively large policy updates.

If the new policy changes too much, PPO ignores the extra improvement. It prefers many small, reliable updates over one large, risky update.

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Evolution of Policy-Gradient Methods

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Method	Uses	Update	Variance
REINFORCE	Monte Carlo Return	Episode End	High
Actor-Critic	TD Error	Every Step	Lower
A2C	Advantage Estimate	Every Step	Lower
PPO	Clipped Advantage	Every Step	Lower + Stable

Each algorithm improves the estimate of the policy gradient and makes learning more stable.

Conclusion

Value-Based vs. Policy-Based Methods

Value-Based	Policy-Based
Learn V or Q	Learn π_θ directly
Policy derived by maximization	Policy is the learned object
Natural for discrete actions	Natural for continuous actions
Often lower variance	Often higher variance
Can be off-policy	Often on-policy

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Modern Policy-Gradient Methods

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

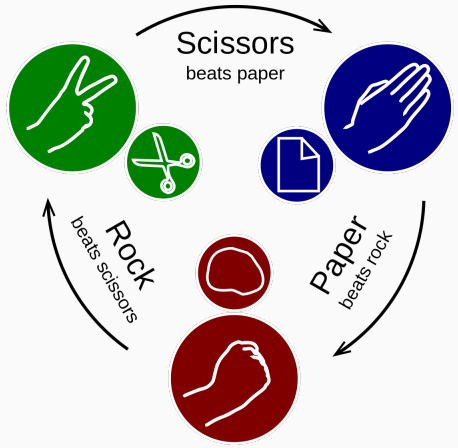
Many modern deep RL algorithms build on this idea:

- REINFORCE
- Actor-Critic
- A2C / A3C
- TRPO
- PPO
- SAC

Policy gradients are central to modern continuous-control RL.

Extras

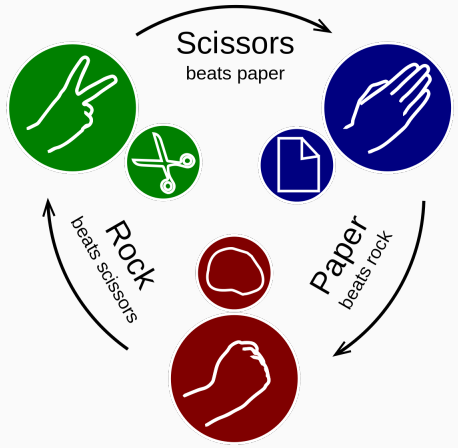
Example: Rock-Paper-Scissors



- Two-player game of rock-paper-scissors
  - Scissors beats paper
  - Rock beats scissors
  - Paper beats rock
- Consider policies for iterated rock-paper-scissors

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

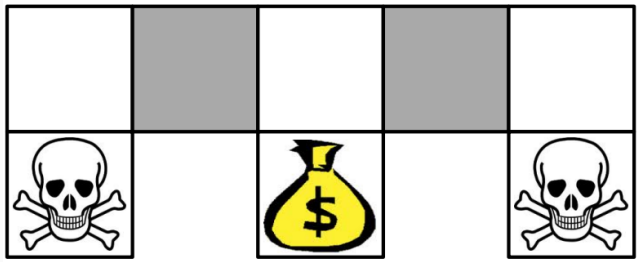
Example: Rock-Paper-Scissors



- Two-player game of rock-paper-scissors
  - Scissors beats paper
  - Rock beats scissors
  - Paper beats rock
- Consider policies for iterated rock-paper-scissors
  - A deterministic policy is easily exploited
  - A uniform random policy is optimal (i.e. Nash equilibrium)

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Example: Aliased Gridworld



- The agent cannot differentiate the grey states
- Consider features of the following form (for all N, E, S, W)

$$\phi(s,a) = 1(walltoN, a = moveE)$$

- Compare value-based RL, using an approximate value function

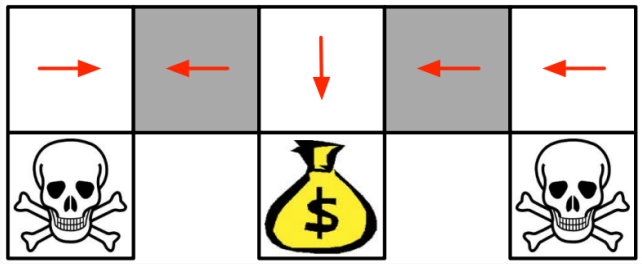
$$Q_\theta(s,a) = f(\phi(s,a),\theta)$$

- To policy-based RL, using a parametrised policy

$$\pi_\theta(s,a) = g(\phi(s,a),\theta)$$

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

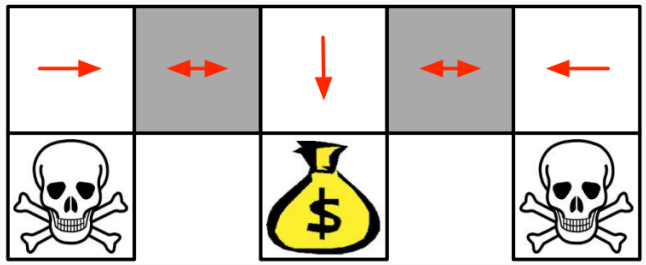
Example: Aliased Gridworld (continued 1)



- Under aliasing, an optimal deterministic policy will either
  - move W in both grey states (shown by red arrows)
  - move E in both grey states
- Either way, it can get stuck and never reach the money
- Value-based RL learns a near-deterministic policy
  - e.g. greedy or ϵ-greedy
- So it will traverse the corridor for a long time

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Example: Aliased Gridworld (continued 2)



- An optimal stochastic policy will randomly move E or W in grey states

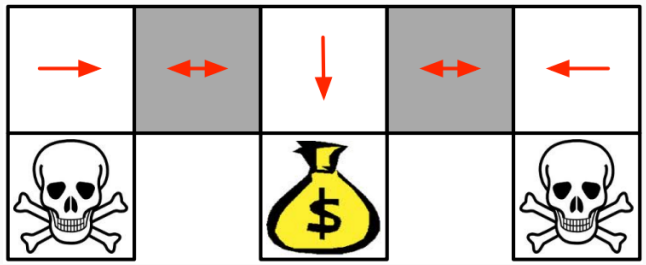
$$\pi_\theta(walltoNandS, moveE) = 0.5$$

$$\pi_\theta(walltoNandS, moveW) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn an optimal stochastic policy

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Example: Aliased Gridworld (continued 2)



- An optimal stochastic policy will randomly move E or W in grey states

$$\pi_\theta(walltoNandS, moveE) = 0.5$$

$$\pi_\theta(walltoNandS, moveW) = 0.5$$

- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn an optimal stochastic policy

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy Optimization

- Policy based reinforcement learning is an optimisation problem
- Find θ that maximises $J(\theta)$
- Some approaches do not use gradient
  - Hill climbing
  - Simplex / amoeba / Nelder Mead
  - Genetic algorithms
- Greater efficiency often possible using gradient
  - Gradient descent
  - Conjugate gradient
  - Quasi-newton
- We focus on gradient descent, many extensions possible
- And on methods that exploit sequential structure

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Computing Gradients By Finite Differences

- To evaluate policy gradient of $\pi_\theta(s, a)$
- For each dimension $k \in [1, n]$
  - Estimate kth partial derivative of objective function w.r.t. θ
  - By perturbing θ by small amount ϵ in kth dimension

$$\frac{\partial J(\theta)}{\partial \theta_k} \approx \frac{J(\theta + \epsilon u_k) - J(\theta)}{\epsilon}$$

where u_k is unit vector with 1 in kth component, 0 elsewhere

- Uses n evaluations to compute policy gradient in n dimensions
- Simple, noisy, inefficient - but sometimes effective
- Works for arbitrary policies, even if policy is not differentiable

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Policy-Based Reinforcement Learning

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

- So far, we approximated the value or action-value function using parameters θ

$$V_\theta(s) \approx V^\pi(s)$$

$$Q_\theta(s) \approx Q^\pi(s)$$

- A policy was generated directly from the value function
  - e.g. using ϵ-greedy
- An alternative is to directly parametrise the policy $\pi_\theta(s, a) = \mathcal{P}[a|s, \theta]$
- This is (again) model-free reinforcement learning

Advantages of Policy-Based RL

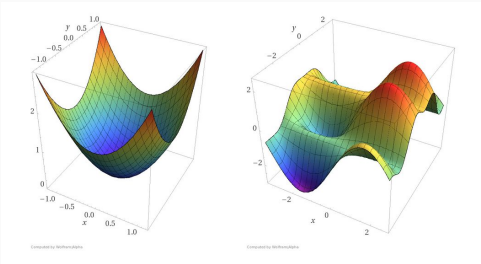
Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

Advantages:

- Better convergence properties
- Effective in high-dimensional or continuous action spaces
- Can learn stochastic policies

Disadvantages:

- Typically converge to a local rather than global optimum
- Evaluating a policy is typically inefficient and high variance



Policy Objective Functions

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

- Goal: given policy $\pi_\theta(s, a)$ with parameters θ, find best θ
- But how do we measure the quality of a policy π_θ?

Policy Objective Functions

Gradient Descent
Policy-based RL
Policy Gradient
REINFORCE
Actor-Critic
Conclusion
Extras

- Goal: given policy $\pi_\theta(s, a)$ with parameters θ, find best θ
- But how do we measure the quality of a policy π_θ?
  - In episodic environments we can use the start value
  - In continuing environments we can use the average value or average reward per time-step